Name: Indrajeet singh

RollNo: 09

Batch: T11

Experiment - 03

Aim: To Perform various GIT operations on local and Remote repositories

Theory:

The command `mkdir git` creates a new directory (folder) named "git" in the current working directory. This command is used to make a new directory in a Unix-like operating system.

The command `cd git` is used to change the current working directory to the directory named "git." After executing this command, any subsequent commands or file operations will occur within the "git" directory. "cd" stands for "change directory."

V

The `git config --global user.name` and `git config --global user.email` commands are used to set

your global Git username and email address, respectively. They are part of the configuration settings in Git and are associated with the commits you make.

```
MINGW64:/c/Users/john/repos/project-x

john@503-015 MINGW64 ~/repos/project-x
$ git log
```

git

If you want to check your configuration settings, you can use the git config --list command to list all the settings Git can find at that point

```
Lenovo@503-025 MINGW64 ~/Documents/python files/git-dvcs/git-demo-
project (master)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Maneesh
user.email=labai2301003@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://github.com/labai2301003-spec/SEPM_T11_03.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*

Lenovo@503-025 MINGW64 ~/Documents/python files/git-dvcs/git-demo-project (master)
$
```

git commit -am "commit message"` stages and commits all changes in tracked files with a commit message in a single command.

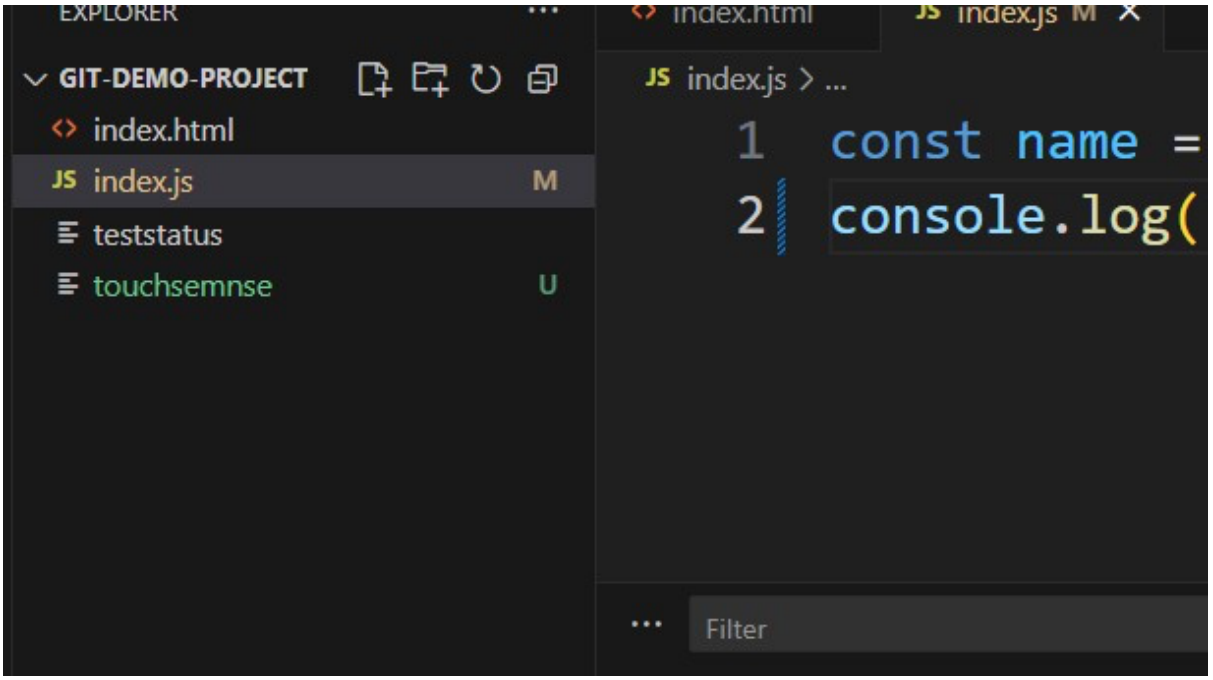
```
Lenovo@503-025 MINGW64 ~/Documents/python files/git-dvcs/git-demo-
project (master)
$ git commit -am "Updated python file"
warning: in the working copy of 'sample_program.py', LF will be re
placed by CRLF the next time Git touches it
[master df91be4] Updated python file
 1 file changed, 1 insertion(+)
```

The command `nano index.html` opens the Nano text editor for the file named "index.html." Nano is a simple command-line text editor that allows you to view and edit files directly in the terminal.

```
MINGW64/c/Users/503/git-dvcs/git-demo-project
  GNU nano 8.5                                                    index.html
<!DOCTYPE html>
<html lang="en">
<head>
-   <meta charset="UTF-8">
-   <meta name="viewport" content="width=device-width, initial-scale=1.0">
-   <title>My First HTML Page</title>
</head>
<body>
-   <h1>Welcome to My Page</h1>
-   <p>This is a basic HTML file with a heading and a paragraph.</p>
-   <ul>
-       <li>Item One</li>
-       <li>Item Two</li>
-       <li>Item Three</li>
-   </ul>
</h1><a href="https://www.example.com">Visit Example.com</a>
</body>
$   ^G Get Help   ^O Write Out   ^R Read File   ^X Exit   ^J Justify   ^W Where Is   ^V Next   Page K
```

The command `touch teststatus` creates an empty file named "teststatus" in the ctouurrent directory. The `touch` command is commonly used to update the timestamps of a file or create an empty file if it doesn't exist.

```
503@503-010 MINGW64 ~/git-dvcs/git-demo-project (master)
$ touch touchsemnse
```



git checkout -- teststatus: Discards changes to the file "teststatus" in the working directory.This reverts the file to the state it has in the last commit.

The `git add` command is used to stage changes in the working directory for the next commit

in Git. It prepares modifications, additions, or deletions to be included in the upcoming commit.

The `git log` command is used to display the commit history of a Git repository. It shows a chronological list of commits, including commit hashes, author information, timestamps, and

commit messages.

The command `git log --oneline` displays a simplified and concise one-line representation of the commit history in a Git repository, showing only the commit SHA-1 hash and the commit message.

```
Lenovo@503-025 MINGW64 ~/projects/api-server-repo  (main)
$ git log
 commit: fe1c5d03d5afb847b9e688e7c8c3a5ad106243be (HEAD -> main)
    Author: Sarah <sarahdev123@example.com>
    Date: Wed Jan 24 14:05:10 2028 +0530
     Add user authentication feature.

 commit: 923d8c77afba2dcd4e1a17fcd828e8fc1226fe980
    Author: Sarah <sarahdev123@example.com>
    Date: Wed Jan 24 13:42:28 2028 +0530
     Initialize API server project
lenovo@503-025 MINGW64 ~/projects/api-server-repo  (main)
```

The `git clone` command is used to create a copy of a Git repository. When you run this command, it duplicates the entire repository, including its files, commit history, and branches, and downloads it to your local machine. This is often the initial step when you want to work with a project hosted on a remote Git repository.

The `git pull` command is used to fetch and integrate changes from a remote repository into the current branch of your local repository. It combines two actions: it fetches the changes from the remote repository, and then it automatically merges those changes into your local branch. This is a convenient way to update your local repository with the latest changes

from the remote repository.

The `git push` command is used to upload or push the local changes in your Git repository to a remote repository. It updates the remote repository with the latest changes made in your local branch, making them accessible to others who share the same remote repository.

The `git fetch` command is used to retrieve changes from a remote repository. It fetches any new branches or changes made in the remote repository since your last interaction. However, it does not automatically merge these changes into your local branches. After using `git fetch`, you can inspect the changes and decide whether to integrate them using `git merge` or `git rebase`.